



Figure 3.5 Seven-segment LED display and hexadecimal patterns.

3.7 DESIGN EXAMPLES

3.7.1 Hexadecimal digit to seven-segment LED decoder

The sketch of a seven-segment LED display is shown in Figure 3.5(a). It consists of seven LED bars and a single round LED decimal point. On the prototyping board, the seven-segment LED is configured as active low, which means that an LED segment is lit if the corresponding control signal is '0'.

A hexadecimal digit to seven-segment LED decoder treats a 4-bit input as a hexadecimal digit and generates appropriate LED patterns, as shown in Figure 3.5(b). For completeness, we assume that there is also a 1-bit input, *dp*, which is connected directly to the decimal point LED. The LED control signals, *dp*, *a*, *b*, *c*, *d*, *e*, *f*, and *g*, are grouped together as a single 8-bit signal, *sseg*. The code is shown in Listing 3.12. It uses one selected signal assignment statement to list all the desired patterns for the seven LSBs of the *sseg* signal. The MSB is connected to *dp*.

Listing 3.12 Hexadecimal digit to seven-segment LED decoder

```

library ieee;
use ieee.std_logic_1164.all;
entity hex_to_sseg is
  port(
    5   hex: in std_logic_vector(3 downto 0);
        dp: in std_logic;
        sseg: out std_logic_vector(7 downto 0)
  );
end hex_to_sseg;
10
architecture arch of hex_to_sseg is
begin
  with hex select
    sseg(6 downto 0) <=
    15   "0000001" when "0000",
        "1001111" when "0001",

```

```

                "0010010" when "0010",
                "0000110" when "0011",
                "1001100" when "0100",
20         "0100100" when "0101",
                "0100000" when "0110",
                "0001111" when "0111",
                "0000000" when "1000",
                "0000100" when "1001",
25         "0001000" when "1010", --a
                "1100000" when "1011", --b
                "0110001" when "1100", --c
                "1000010" when "1101", --d
                "0110000" when "1110", --e
30         "0111000" when others; --f
    sseg(7) <= dp;
end arch;

```

There are four seven-segment LED displays on the prototyping board. To save the number of FPGA chip's I/O pins, a time-multiplexing scheme is used. The block diagram of the time-multiplexing module, `disp_mux`, is shown in Figure 3.6(a). The inputs are `in0`, `in1`, `in2`, and `in3`, which correspond to four 8-bit seven-segment LED patterns, and the outputs are `an`, which is a 4-bit signal that enables the four displays individually, and `sseg`, which is the shared 8-bit signal that controls the eight LED segments. The circuit generates a properly timed enable signal and routes the four input patterns to the output alternatively. The design of this module is discussed in Chapter 4. For now, we just treat it as a black box that takes four seven-segment LED patterns, and instantiate it in the code.

Testing circuit We use a simple 8-bit increment circuit to verify operation of the decoder. The sketch is shown in Figure 3.6(b). The `sw` input is the 8-bit switch of the prototyping board. It is fed to an incrementor to obtain `sw+1`. The original and incremented `sw` signals are then passed to four decoders to display the four hexadecimal digits on seven-segment LED displays. The code is shown in Listing 3.13.

Listing 3.13 Hex-to-LED decoder testing circuit

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity hex_to_sseg_test is
5   port (
        clk: in std_logic;
        sw: in std_logic_vector(7 downto 0);
        an: out std_logic_vector(3 downto 0);
        sseg: out std_logic_vector(7 downto 0)
10    );
end hex_to_sseg_test;

architecture arch of hex_to_sseg_test is
    signal inc: std_logic_vector(7 downto 0);
    signal led3, led2, led1, led0: std_logic_vector(7 downto 0);
15   begin
        -- increment input
        inc <= std_logic_vector(unsigned(sw) + 1);

```